

BSDA2001P: Introduction to DL and GenAI Project

Smart MCQ Solver Challenge



Kalyan Dath

21f2000763

GitHub Repo: <https://github.com/KalyanDath/dl-genai-project-26-t2>

Weights & Biases Project: <https://wandb.ai/21f2000763-dl-genai-project/21f2000763-t22026>

Streamlit App: [Streamlit](#)

1. Abstract

The problem statement involves creating a model using Deep Learning and Generative AI techniques to predict the correct option among the set of 5 options given the matches as the correct answer to the corresponding question. Each question contains a prompt along with 5 possible answer options labeled A, B, C, D, E.



The objective is to predict the top three most likely correct answer in the ranked order.

The challenge submission will be evaluated using Mean Average Precision at 3 (MAP@3) which rewards the models that rank the correct answer higher in their predictions.

The project successfully developed and tracked three distinct machine learning architectures to address this challenge:

- **Model 1 (From Scratch):** A custom Bidirectional LSTM with an Attention Mechanism built entirely in PyTorch.
- **Model 2 (Pre-trained / Context Retrieval):** An advanced Retrieval-Augmented Generation (RAG) pipeline utilizing FAISS vector indexing, a HuggingFace Cross-Encoder for reranking, and a 4-bit quantized **Qwen2.5-7B-Instruct** Large Language Model.
- **Model 3:** A fine-tuned **DeBERTa-v3-base** Transformer utilizing a specialized Multiple Choice data collator.

2. Introduction

Problem Statement

The core objective of the Kaggle competition is to answer multiple-choice questions by predicting a ranked list of the top 3 option keys (e.g., A, C, B).

3. Dataset & Preprocessing

Dataset Description

The project utilizes the primary competition data (train.csv and test.csv) containing prompts and five candidate options (A-E). Additionally, external knowledge bases (physics_pages.csv and Wiki_knowledge_raw.csv) were integrated to construct the RAG vector store.

Exploratory Data Analysis (EDA)

Initial EDA revealed the presence of duplicate prompts within the training data, which were programmatically dropped to prevent data leakage and overfitting. Character and word counts were analyzed to determine optimal maximum sequence lengths for tokenization (settling on `max_length` = 256 for most models).

The ground-truth answers (A, B, C, D, E) were analyzed for class imbalances. The frequency distribution confirmed a relatively balanced dataset, allowing us to accurately rely on standard ranking metrics without skewing predictions toward a heavily dominant single class.

Text Normalization and Cleaning

For the RAG knowledge base, a robust text cleaning pipeline was implemented using regular expressions to strip noisy data from the raw scraped pages. The cleaning function specifically targeted:

- Wikipedia citation markers (e.g., [1], [23])
- Section headers (e.g., === History ===)
- LaTeX formatting and mathematical commands ($\frac{1}{2}$, α)
- Page number artifacts.

Example of the regex cleaning pipeline for RAG context

```
text = re.sub(r'\\[\\d+\\]\\s*', "", text)

text = re.sub(r'^={2,}.*?={2,}', "", text, flags=re.MULTILINE)

text = re.sub(r'\\[a-zA-Z]+', " ", text)

text = "\\n".join(line.strip() for line in text.splitlines())
```

4. Tokenization & Feature Extraction Strategy

Multiple Choice Formatting

To evaluate multiple choices effectively, sequence models require the question and the options to be jointly encoded. Two distinct formatting strategies were employed depending on the model architecture:

- **For the BiLSTM:** The prompt and all five options were concatenated into a single string separated by Special Tokens: `[Prompt] [SEP] A: [Option A] B: [Option B] ...`
- **For DeBERTa:** The dataset was expanded into 5 candidate pairs per row, concatenating the prompt with each individual option choice iteratively to generate a flat input tensor of shape `(batch_size, 5, sequence_length)`.

Embedding & Vectorization

For the RAG pipeline, texts were split using the **RecursiveCharacterTextSplitter** (chunk size 1000, overlap 300). These chunks were vectorized using the **BAAI/bge-large-en-v1.5** HuggingFace embeddings and stored in a local FAISS index for high-speed similarity search.

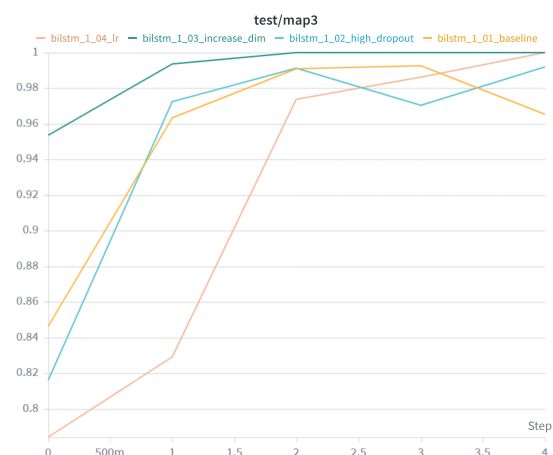
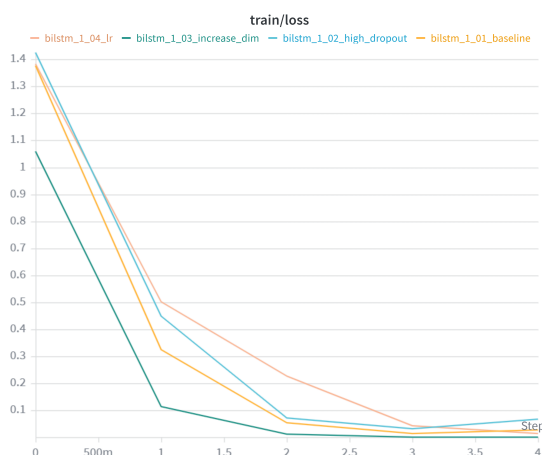
5. Modelling & Experimentation

Model 1: Custom BiLSTM with Attention

Architecture: A custom PyTorch model utilizing pre-trained tokenizers (google-bert/bert-base-uncased) to feed embedding layers. The core sequence processing is handled by a Bidirectional LSTM.

Salient Points: To focus on the most relevant parts of the question-option sequence, a custom Attention layer computes weighting scores for the LSTM hidden states. The masked softmax outputs form a context vector that is passed through a fully connected layer with Dropout to output logits for the 5 classes.

Hyperparameter	Value
Embedding & Hidden Dim	256
Dropout	0.5
Learning Rate	1e-3
Batch Size	16



Model 2: RAG Pipeline with Qwen2.5-7B-Instruct

Architecture: An advanced Generative AI inference pipeline. The model uses a two-stage retrieval process: a base retriever fetches 15 documents from the FAISS index, which are then passed to a Cross-Encoder (**ms-marco-MiniLM-L-6-v2**) to rerank and compress the context down to the top 3 most relevant passages.

Salient Points: To manage GPU VRAM constraints, the Qwen2.5-7B LLM was loaded using 4-bit NormalFloat (NF4) quantization via **BitsAndBytesConfig**. A strict XML-enforced Prompt Template ensures the LLM outputs exactly three ranked characters without conversational hallucination.

```
prompt_template='''
    You are an expert scientific multiple-choice question answering assistant.
    ant.
```

Use ONLY the provided context to answer.

Rules:

- Output ONLY one XML tag.
- Inside the tag, write exactly THREE answer letters ranked from most likely to least likely.
- Use only the letters A, B, C, D, and E.
- Do not explain your reasoning.
- Do not output any text before or after the XML tag.
- Example xml format: <answer>A C E</answer>

Context:

{context}

Question:

{question}

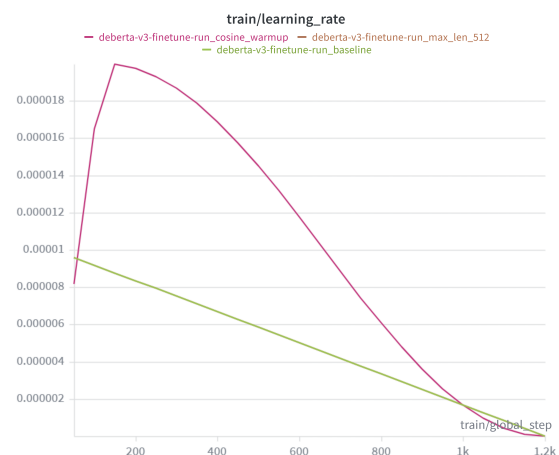
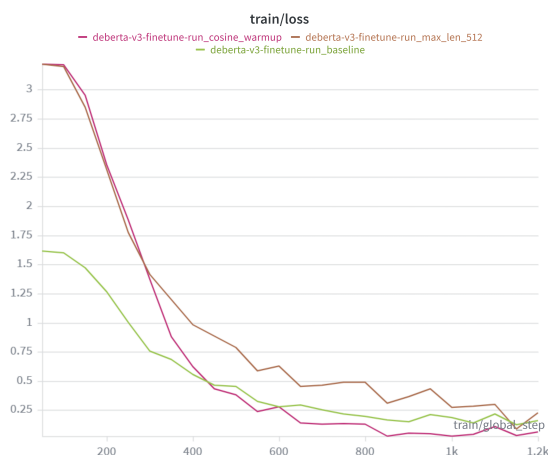
Answer:

...

Model 3: Fine-Tuned DeBERTa-v3-base

Architecture: `microsoft/deberta-v3-base` leveraging the `AutoModelForMultipleChoice` head.

Salient Points: `DeBERTa` improves upon standard BERT architectures by using disentangled attention mechanisms, making it highly effective at semantic similarity between prompts and options. The model was fine-tuned using the HuggingFace Trainer API.



6. Performance & Comparative Analysis

Evaluation Metric (`mAP@3`)

Models were evaluated using a custom `map3_score` function during validation. Mean Average Precision at 3 rewards models that place the correct ground-truth option higher in the predicted top-3 sequence.

Comparative Results

Model Architecture	Strengths	Score
BiLSTM with Attention	Fast training iteration, lightweight inference, zero external dependencies.	0.75477
Qwen RAG Pipeline	Highly accurate on factual queries; zero-shot capability requires no fine-tuning.	0.71778
DeBERTa-v3 Fine-Tuned	Excellent balance of inference speed and contextual understanding.	0.76018

7. Conclusion & Future Work

Key Learnings

The project successfully demonstrated the end-to-end pipeline required for modern NLP tasks. Implementing 4-bit quantization for the RAG LLM and utilizing a custom `DataCollatorForMultipleChoice` for DeBERTa allowed for complex model training within constrained environments.

Future Work

To further improve the mAP@3 score, future iterations will focus on:

- Ensembling:** Extracting the logits from say, both the DeBERTa model and the BiLSTM to create a weighted voting ensemble.
- LoRA Fine-Tuning:** Applying Parameter-Efficient Fine-Tuning (PEFT) to the Qwen LLM to adapt its generative style more closely to the scientific MCQ format.